GAME DESIGN DOCUMENT

Rock · Paper · Scissors

The Ancient Duel

Studio	Solo / Portfolio Project
Genre	Arcade / Casual
Platform	Desktop (Windows, macOS, Linux)
Engine	SDL2 + C++17
Version	1.0 — Initial Release
Date	June 2021

1. Game Overview

1.1 Concept Statement

Rock Paper Scissors is a fast-paced 2D arcade game that digitises the classic hand-game into a polished desktop experience. The player chooses from three hand gestures — Rock, Paper, or Scissors — competing against a randomised CPU opponent. Each round features a satisfying throw animation and a dramatic reveal, turning a simple decision into a tense micro-duel.

1.2 Design Pillars

- Instant Gratification — every round resolves in under three seconds; there is zero downtime.
  - Visual Clarity — hand shapes are immediately readable; the outcome is never ambiguous.
  - Replayability — the RNG opponent and persistent scoring create a compelling 'one more round' loop.
  - Tactile Feel — animations and visual feedback make each input feel weighty and satisfying.

1.3 Target Audience

Primary: Casual desktop gamers, all ages. Secondary: Developers and designers browsing portfolio work who want to see an end-to-end SDL2 project.

1.4 Unique Selling Points

- Hand-drawn SDL2 primitives — no sprite sheets; all visuals are procedurally drawn in C++.
  - Animated throw sequence — a physics-inspired sine-wave bounce precedes every reveal.
  - Zero dependencies beyond SDL2 and SDL2_ttf — compiles on any platform in one line.

2. Gameplay

2.1 Core Loop

1. Player views the arena and selects a gesture by clicking one of three buttons.
2. An animation plays: the player's hand bounces (sine-wave, ~1.8 s) while the CPU panel shows a '?' placeholder.
3. The CPU's choice is randomly selected (uniform 33.3% distribution) and revealed via a fade-in.
4. The outcome (Win / Lose / Tie) is determined and displayed with colour-coded text.
5. The persistent scoreboard increments accordingly.
6. The player may immediately start the next round.

2.2 Win Conditions

Player Choice	CPU Choice	Result
Rock	Scissors	WIN — Rock crushes Scissors
Paper	Rock	WIN — Paper covers Rock
Scissors	Paper	WIN — Scissors cuts Paper
Rock	Paper	LOSE
Paper	Scissors	LOSE
Scissors	Rock	LOSE
Any	Same	TIE

2.3 Scoring

- Player wins a round: +1 to Player score.
- CPU wins a round: +1 to CPU score.
- Ties: no score change.
- Scores persist across rounds; right-click resets both to zero.
- There is no match limit in v1.0 — the session continues indefinitely.

2.4 RNG Design

The CPU selects its gesture using C's rand() seeded with the Unix timestamp at launch (srand(time(nullptr))). Each choice has equal 1-in-3 probability. Future versions may introduce weighted personalities (see Section 7).

3. Controls & Input

3.1 Input Map

Input	Action
Left-click Rock button	Select Rock — begins throw animation
Left-click Paper button	Select Paper — begins throw animation
Left-click Scissors button	Select Scissors — begins throw animation
Right-click (anywhere)	Reset both scores to 0
Escape / Window Close	Quit the application

3.2 Button States

- Idle — default dark background with violet border.
- Hover — lighter fill; cursor implicitly changes via OS pointer.
- Pressed — brightest fill; visual confirmation of click.
- Disabled — buttons are non-interactive during the animation phase to prevent input spam.

4. Art & Visual Design

4.1 Visual Style

The aesthetic is dark, refined arcade — a deep navy/indigo palette with vivid violet accents and cyan/amber highlights. All graphics are drawn programmatically using SDL2 rendering primitives (lines, filled circles, rectangles). There are no external sprite or texture assets.

4.2 Colour Palette

Role	Hex	Usage
Background	#0F0C1E	Window fill
Panel	#1C1837	Arena and button backgrounds
Accent / Violet	#8250FF	Borders, active states
Text Primary	#F0EBFF	Headings and labels
Text Dim	#8C82B4	Subtitles, hints
Win Green	#50E68C	Win outcome text
Lose Red	#E64650	Lose outcome text
Tie Amber	#FAC83C	Tie outcome text
Skin (hands)	#F5C878	Hand fill
Skin Shadow	#C8923A	Knuckle and crease lines

4.3 Hand Geometry

Rock

- Large filled ellipse (62 × 56 px) for the fist body.
- Four rounded knuckle ellipses along the top edge.
- Crease lines rendered as 2 px dark-skin strokes.
- Thumb: offset ellipse on the left side.

Paper

- Wide rectangular palm (74 × 68 px) with rounded corners.
- Four upward finger rectangles with rounded caps.
- Thumbnail clipped ellipses for fingernail highlights.
- Lateral thumb on left edge.

Scissors

- Palm identical to Paper base.
- Two spread fingers rendered with per-row horizontal shift to simulate a V-angle rotation.

- Ring and pinky fingers shown as folded nubs.
- Scissor spread angle driven by a configurable spread constant (default ±8 px at fingertip).

4.4 Animation

The throw animation runs for 1.8 seconds at 60 FPS. Vertical displacement is calculated as:

```
offset_y = sin(progress × 3.5 × 2π) × 28 × (1 − progress × 0.6)
```

This produces a dampening bounce that settles naturally. The CPU panel displays a '?' glyph during this phase and then fades in its hand over 0.6 seconds using an SDL alpha overlay.

5. Audio Design

5.1 v1.0 Status

Audio is out of scope for the initial SDL2 release. The implementation intentionally avoids SDL2_mixer to keep the dependency list minimal and the build process single-command.

5.2 Planned Sound Events (v1.1)

- Button hover — soft tick (< 50 ms, 440 Hz sine).
- Button press / throw start — quick whoosh downward sweep.
- Reveal — short 'pop' or 'snap' (choice-dependent: thud for Rock, rustle for Paper, snip for Scissors).
- Win — ascending two-tone chime.
- Lose — descending minor interval.
- Tie — neutral single-note ping.
- Score reset — short rewind-style effect.

5.3 Audio Style

Procedurally generated tones using SDL2_mixer's raw audio callback, matching the no-asset philosophy. All sounds synthesised from sine/sawtooth waves and ADSR envelopes in < 200 lines of C++.

6. Technical Specification

6.1 Architecture

- Single-file implementation (rps_game.cpp) — all game logic, rendering, and state management in one translation unit for portfolio clarity.
- Game loop runs at a fixed 60 FPS via SDL_Delay(1000/FPS) — no delta-time interpolation required at this scale.
- State machine: IDLE → ANIMATING → RESULT → (back to IDLE on next input).
- All geometry functions receive a float scale parameter for DPI-agnostic sizing (future use).

6.2 Build Requirements

Language	C++17
Renderer	SDL2 (Accelerated, VSync)
Text	SDL2_ttf
Min Resolution	800 × 600 px
Target FPS	60
Build (Unix)	g++ rps_game.cpp -o rps_game $(sdl2-config --cflags --libs) -lSDL2_ttf -std=c++17
Build (Win)	cl rps_game.cpp /link SDL2.lib SDL2_ttf.lib SDL2main.lib

6.3 Font Handling

The game attempts to open DejaVu Sans from common system paths. On macOS it falls back to Arial. A final fallback to Liberation Sans is provided. All three font sizes (32 pt title, 20 pt body, 15 pt small) are opened as separate TTF_Font handles and destroyed on exit.

6.4 Performance Budget

- CPU: < 2% on any modern desktop at 60 FPS (all rendering is 2D primitive-only).
- Memory: < 8 MB total process footprint.
- Launch time: < 0.5 s cold start.

7. Future Development

7.1 v1.1 — Audio

- Integrate SDL2_mixer for synthesised sound effects (see Section 5).
- Optional background music: procedural lo-fi beat.

7.2 v1.2 — Best-of Mode

- Add a match system: Best of 3 / 5 / 7, selectable from a start screen.
- Match winner screen with animated confetti (SDL-drawn circles).
- Match history log displayed in a scrollable panel.

7.3 v1.3 — CPU Personalities

- Introduce three CPU difficulty modes:
  - Random (current) — pure uniform RNG.
  - Adaptive — tracks player's last 5 choices and biases counter-selection.
  - Tells — occasionally 'telegraphs' its choice with a subtle pre-animation.

7.4 v2.0 — Network Multiplayer

- Local-area network 1v1 via UDP sockets (SDL_net).
- Simultaneous reveal — both players lock in before either is shown.
- Simple lobby system with room codes.

7.5 Stretch Goals

- Controller support via SDL2 GameController API.
- High-score leaderboard persisted to a local JSON file.
- Accessibility: keyboard-only navigation (Tab to cycle buttons, Enter to select).
- Linux Flatpak / Windows installer packaging.

Appendix A — File Structure

```
rps_game/  rps_game.cpp      — Full game source (single file)  GDD.docx
— This document  poster.svg         — 2D promotional art poster  README.md
— Build & run instructions  LICENSE            — MIT Licence
```

Appendix B — Glossary

Arena Panel The framed region on either side of the screen where hands are displayed.

Throw Animation The sine-wave bounce sequence that plays after the player selects a gesture.

Reveal The CPU's hand fade-in that occurs after the throw animation completes.

State Machine The finite set of game states: IDLE, ANIMATING, RESULT.

DXA Document eXchange Arbitrary units — 1/1440th of an inch, used in OOXML.

SDL2 Simple DirectMedia Layer version 2 — a cross-platform multimedia library.

TTF TrueType Font — the font format loaded via SDL2_ttf.

Appendix C — Revision History

Version	Date	Author	Notes
1.0	June 2021	Portfolio	Initial GDD — covers core gameplay, visuals, tech spec